

Arquitetura Multi-DeepNet para Localização com Baixo SNR e Poucos Snapshots

Rede Neural Convolucional

Paulo Rogério Alfredo Candido Junior

Department of Electrical Engineering.
State University of Londrina

March 27, 2025

- 1 Descrição do Problema
- 2 Aquisição dos dados sintéticos
- 3 Arquitetura da rede Multi-DeepNET

Objetivo; realizar a estimativa simultânea do ângulo de chegada de múltiplas fontes que transmitem poucos miliwatts utilizando poucos snapshots.

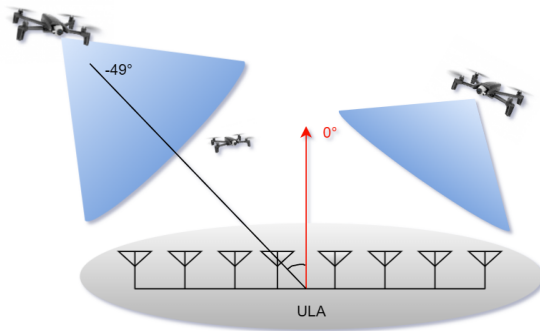
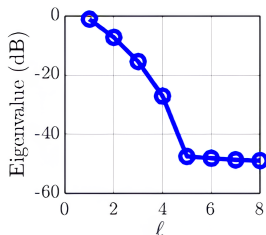


Figura: Sistema MIMO em cenário com linha de visada (LoS), modelo espaço livre com múltiplas fontes.

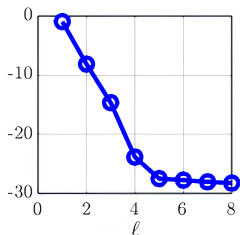
Técnicas tradicionais como MUSIC apresentam degradação expressiva de desempenho devido a sensibilidade ao ruído durante a decomposição espectral (*threshold effect*). Logo torna-se desafiador realizar o sensoriamento para;

- Condição severa de baixa latência, (apenas um snapshot)
- Fontes que transmitem poucos miliwatts. (IoT)
- Cenário com SNR muito baixa

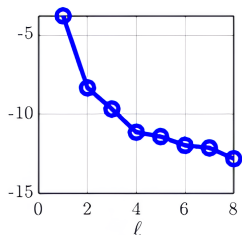
Por mais que o sistema seja determinístico devido a linha de visada (LoS), é de difícil inversão, altamente não linear, com ambiguidade e interferência de ruído.



(a) $\sigma_n^2 = -90$ dBm



(b) $\sigma_n^2 = -70$ dBm



(c) $\sigma_n^2 = -50$ dBm

Figura: Diferença entre autovalores consecutivos para 8 antenas, com 4 fontes. Para uma SNR elevada, podemos realizar essa estimativa de forma simples, através da separação natural entre os autovetores.

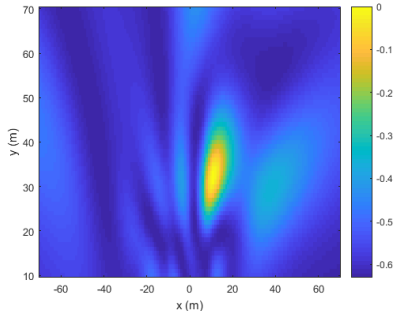
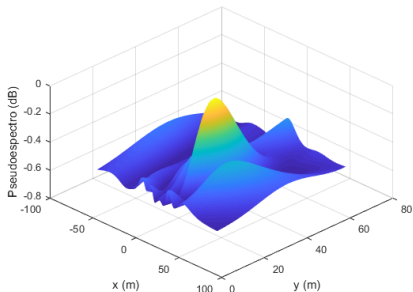


Figura: o pseudospectro MUSIC (função objetivo) obtido a partir do subespaço de ruído mostra uma resposta complexa e não convexa, para uma SNR de 3dB e uma única fonte. Embora exista um único máximo global que representa a posição correta da fonte, a presença de máximos locais e a topologia ondulada da função dificultam a inversão direta do modelo, principalmente em ambientes ruidosos.

A solução proposta é baseada em aprendizagem supervisionada usando *Deep Learning*, adaptando o método Multi-DeepNet originalmente desenvolvido para radares FMCW-MIMO (Su et al., 2023), especificamente para o contexto de localização multiusuário com um conjunto de antenas linear (ULA). A abordagem atende diretamente a necessidade de soluções robustas sob baixa SNR, necessidade de baixa latência e multiuser.



Su, Yunye and Lan, Xiang and Shi, Jinmei and Sun, Lu and Wang, Xianpeng

Fast target localization in FMCW-MIMO radar with low SNR and snapshot via multi-DeepNet.

Remote Sensing, vol. 15, no. 1, pp. 66, 2022.

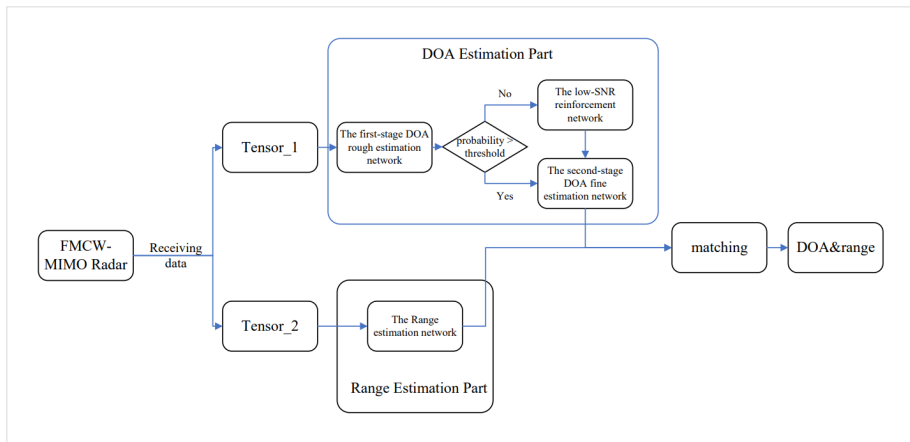


Figura: O processo completo de estimativa do Multi-DeepNet (Su et al., 2023).

O projeto foi limitado na reprodução da etapa inicial do artigo base; classificar a região ampla onde os usuários estão situados, fornecendo uma estimativa grosseira. Na saída teremos um vetor de probabilidades com $N = 12$ classes representando a distribuição de confiança sobre as regiões.

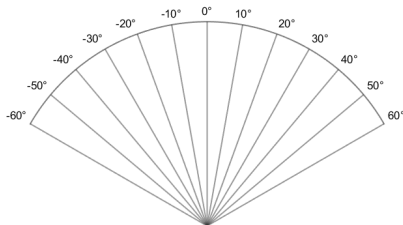


Figura: Busca grosseira primeira etapa.

- 1 Descrição do Problema
- 2 Aquisição dos dados sintéticos**
- 3 Arquitetura da rede Multi-DeepNET

Os dados foram gerados de forma sintética utilizando MCS. Para isso foi necessário definir o modelo de sinal recebido e valida-lo via simulação. Cada fonte transmite um sinal .Para modelar os efeitos da perda de percurso com $\alpha = 2$, usamos o modelo;

$$\beta = \frac{A_{\text{iso}}}{A_{\text{sphere}}} = \frac{\frac{\lambda}{4\pi}}{4\pi d^2} = \frac{3\text{Ghz}}{(4\pi)^2 f} \left(\frac{1}{d}\right)^2 = \Upsilon \left(\frac{1}{d}\right)^2$$

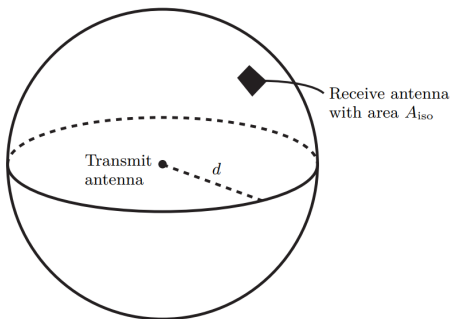


Figura: Perda de percurso do sinal irradiado por uma antena isotrópica. Porção de potência captada pelo receptor.

Para atribuir os efeitos da diferença de fase em cada antena ao modelo, utilizamos a aproximação para sinais com frente de onda plana devido ao campo distante.

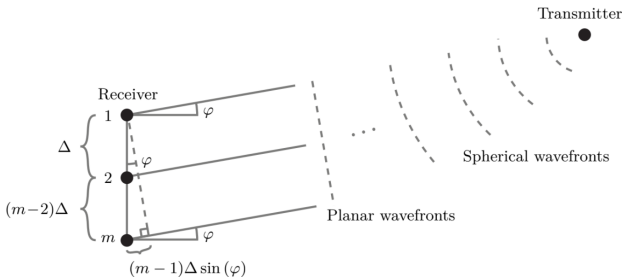


Figura: comportamento da frente de onda para campo distante. Permite realizar aproximações, com base na geometria que surge entre a frente de onda e o ULA.

Dessa forma construímos o vetor de varredura (steering vector);

$$\mathbf{a}_k(\theta_k) = \left[1, e^{-j\frac{2\pi\Delta}{\lambda} \sin(\theta_k)}, e^{-j\frac{4\pi\Delta}{\lambda} \sin(\theta_k)}, \dots, e^{-j\frac{2\pi\Delta}{\lambda} (m-1) \sin(\theta_k)} \right]$$

Dessa forma obtemos uma modelagem útil para representação dos efeitos do canal, onde $y_m(l)$ é a serie temporal amostrada da combinação dos sinais recebidos na m-ésima antena. Incluído o ruído aditivo ao modelo final, $n \sim \mathcal{N}_{\mathbb{C}}(0, \sigma^2)$.

$$y_m(l) = \sum_{k=1}^K \sqrt{\beta_k} s_k(l) e^{-j \frac{2\pi \Delta}{\lambda} (M-1) \sin(\alpha)} + n(l)$$

Construída a matriz de sinais recebidos $\mathbf{Y} \in \mathbb{C}^{m \times l}$ a partir de séries temporais de sinais recebidos em cada antena;

$$\mathbf{Y} = \mathbf{A}\mathbf{S} + \mathbf{N}, \quad \mathbf{N} \sim \mathcal{N}_{\mathbb{C}}(0, \sigma^2 \mathbf{I}_M)$$

Com a modelagem de $\mathbf{Y}$ podemos obter informações espaciais relevantes a partir da média da matriz de covariância de $\mathbf{Y}$;

$$\mathbf{R}_{\mathbf{Y}} = \frac{\mathbb{E}[\mathbf{Y}\mathbf{Y}^H]}{l}$$

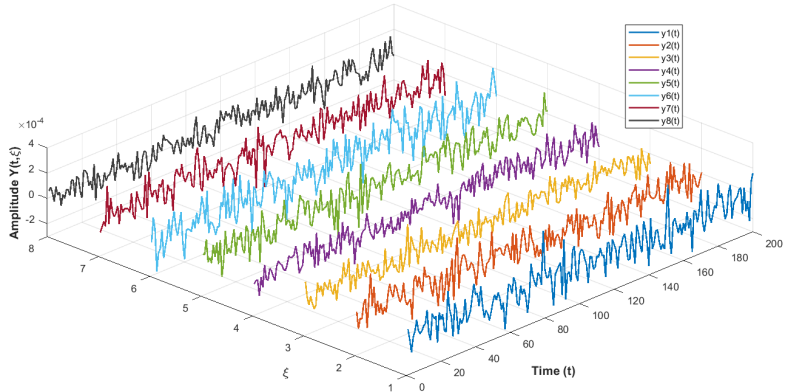


Figura: Séries temporais dos sinais recebidos em cada antena do arranjo. Os sinais recebidos já estão sobre os efeitos do canal.

Para validar a consistência da modelagem de $\mathbf{Y}$ observamos a amostragem contradomínio da função obtida na saída do algoritmo *Multiple Signals Classification (MUSIC)*, utilizando $\mathbf{Y}$.

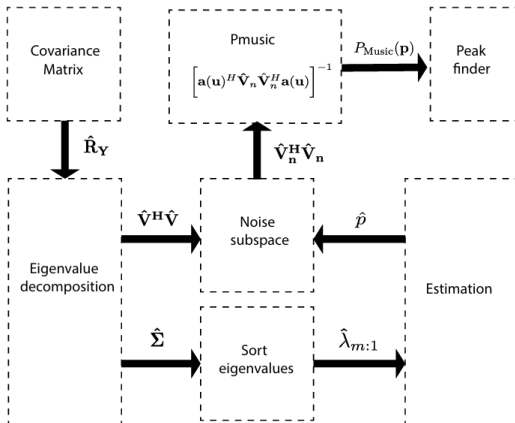


Figura: Diagrama MUSIC

Tabela: PARÂMETROS DO SISTEMA UTILIZADO NO ARTIGO

Parâmetro	Valor
Realizações por SNR	$\mathcal{M} = 5 \cdot 10^5$
Snapshots por amostra	$N = 10$
Número de antenas	$M = 10$ (ULA)
Espaçamento entre elementos	$\delta = \frac{\lambda}{2}$
Comprimento de onda	$\lambda = \frac{c}{f_c}$
Frequência portadora	$f_c = 78,74$ GHz
Potência do sinal transmitido	$\mathcal{P} = 30$ mW
Expoente de perda de percurso	$\alpha = 2$
Intervalo de SNR utilizado	$\text{SNR} \in [-20:2:20]$ dB
Intervalo de ângulos de incidência	$\theta_k \sim \mathcal{U}(-60^\circ, 60^\circ)$
Intervalo de distâncias	$R_k \sim \mathcal{U}(1, 10)$ m
Número de alvos por amostra	$K \sim \mathcal{U}\{1, 6\}$
Quantização angular (coarse)	10° por bin

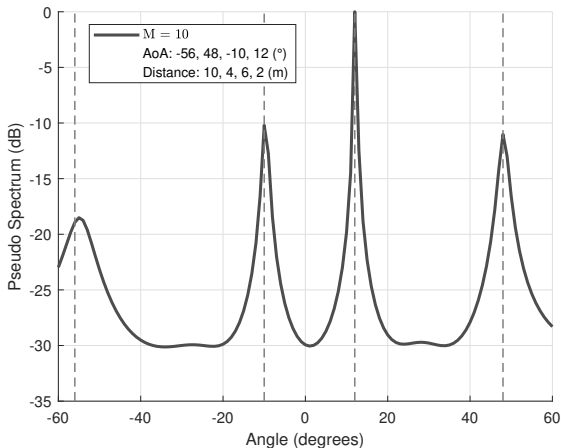


Figura: Pseudo espectro normalizado em decibéis obtido a partir da modelagem dos sinais, onde as linhas verticais tracejadas em cinza indicam os ângulos de chegada (AoA) verdadeiros dos alvos. A coincidência entre os picos do pseudoespectro e os AoAs valida a geração dos sinais recebidos no ULA ($\mathbf{Y}$).

Validado a geração dos dados, é iniciada construção do conjunto de dados rotulado $\mathcal{D} = \{(\mathcal{T}^{(k)}, \mathbf{t}^{(k)})\}_{k=1}^K$, foram geradas amostras sintéticas no MATLAB, simulando um radar FMCW-MIMO com múltiplos alvos em diferentes ângulos entre -60° e 60° e distâncias até 10 metros.

Gerado até 200.000 observações por nível de SNR, variando de -20 dB a $+20$ dB, considerando até 3 usuários simultâneos. Cada observação é um tensor tridimensional $\mathcal{T} \in \mathbb{R}^{10 \times 10 \times 3}$. A saída correspondente $\mathbf{t}$ consiste em vetores de probabilidade para classificação angular (DOA):

Tabela: Descrição das entradas da rede e sua interpretação física.

Entrada	Tipo	Interpretação
$\Re\{\mathbf{R}\}$	Numérico	Parte real da matriz de covariância dos sinais recebidos.
$\Im\{\mathbf{R}\}$	Numérico	Parte imaginária da matriz de covariância.
$\angle \mathbf{R}$	Numérico	Fase dos elementos da matriz de covariância.
DOA	Categórico	Vetor de probabilidade de cada setor angular (12 classes).

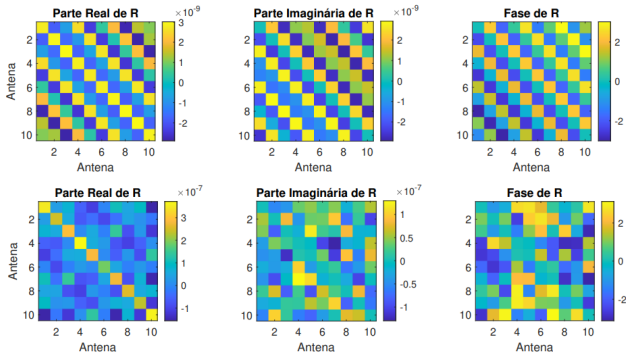


Figura: componentes da matriz de covariancia em $\Re\{\mathbf{R}\}$, $\Im\{\mathbf{R}\}$, $\angle\mathbf{R}$ gerada com $AoA = 45^\circ$. Essas componentes compoem o tensor $\mathcal{T}$

Project - MultiDeepNet-FMCW-MIMO

Views: All **Project (36)** Modified (39)

Layout: Tree

Files

Dependency Analyzer

Labels

Classification

Git

Current branch: master
Branch status: Normal
Ahead of /origin/master

Name	Status	Classification	Git
datasets	✓		•
dataset_coarse_SNR-02.mat	✓	Dataset	●
dataset_coarse_SNR-04.mat	✓	Dataset	●
dataset_coarse_SNR-06.mat	✓	Dataset	●
dataset_coarse_SNR-08.mat	✓	Dataset	●
dataset_coarse_SNR-10.mat	✓	Dataset	●
dataset_coarse_SNR-12.mat	✓	Dataset	●
dataset_coarse_SNR-14.mat	✓	Dataset	●
dataset_coarse_SNR-16.mat	✓	Dataset	●
dataset_coarse_SNR-18.mat	✓	Dataset	●
dataset_coarse_SNR-20.mat	✓	Dataset	●
dataset_coarse_SNR+00.mat	✓	Dataset	●
dataset_coarse_SNR+02.mat	✓	Dataset	●
dataset_coarse_SNR+04.mat	✓	Dataset	●
dataset_coarse_SNR+06.mat	✓	Dataset	●
dataset_coarse_SNR+08.mat	✓	Dataset	●
dataset_coarse_SNR+10.mat	✓	Dataset	●
dataset_coarse_SNR+12.mat	✓	Dataset	●
dataset_coarse_SNR+14.mat	✓	Dataset	●
dataset_coarse_SNR+16.mat	✓	Dataset	●
dataset_coarse_SNR+18.mat	✓	Dataset	●
dataset_coarse_SNR+20.mat	✓	Dataset	●

Details

Figura: Dataset gerado, totalizando uma base de dados com 10,500,000 dados rotulados.

- 1 Descrição do Problema
- 2 Aquisição dos dados sintéticos
- 3 Arquitetura da rede Multi-DeepNET**

Em relação a arquitetura de rede usada no artigo, foram feitas as seguintes alterações, buscando combater o *overfitting*;

- redução do número de filtros na camada convolucional.
- regularização (não especificado no artigo.)
- Adicionadas camadas de dropout na *fully connect*.
- Adicionado *early-stopping* de 8 épocas.

Com o objetivo de reduzir o tempo de treinamento devido a limitação computacional, o problema foi reduzido para estimativa $[\hat{r}_k, \hat{\theta}_k]$ de 6 para 3 usuários simultaneamente.

Datasets	Máx. usuários	SNR	Total de amostras
Original	6	-20:2:20	10,5 milhões
Reduzido	3	-15:2:15	4,2 milhões

Tabela: Redução no número de amostras do dataset.

```

53 # -----
54 # 4. Modelo CoarseCNN
55 # -----
56 class CoarseCNN(nn.Module):
57     def __init__(self, num_classes):
58         super().__init__()
59         self.model = nn.Sequential(
60             nn.Conv2d(3, 128, 3, padding=1),
61             nn.BatchNorm2d(128),
62             nn.LeakyReLU(0.3),
63             #nn.Dropout(0.2),
64             nn.Conv2d(128, 64, 3, padding=1),
65             nn.BatchNorm2d(64),
66             nn.LeakyReLU(0.3),
67             #nn.Dropout(0.2),
68             nn.Conv2d(64, 32, 3, padding=1),
69             nn.BatchNorm2d(32),
70             nn.LeakyReLU(0.3),
71             #nn.Dropout(0.2),
72             nn.Flatten(),
73             nn.Linear(32 * 10 * 10, 1024),
74             nn.ReLU(),
75             nn.Dropout(0.5),
76             nn.Linear(1024, 256),
77             nn.ReLU(),
78             nn.Dropout(0.5),
79             nn.Linear(256, num_classes),
80             nn.Sigmoid()
81         )
82
83     def forward(self, x):
84         return self.model(x)

```

Figura: Arquitetura da rede utilizada.

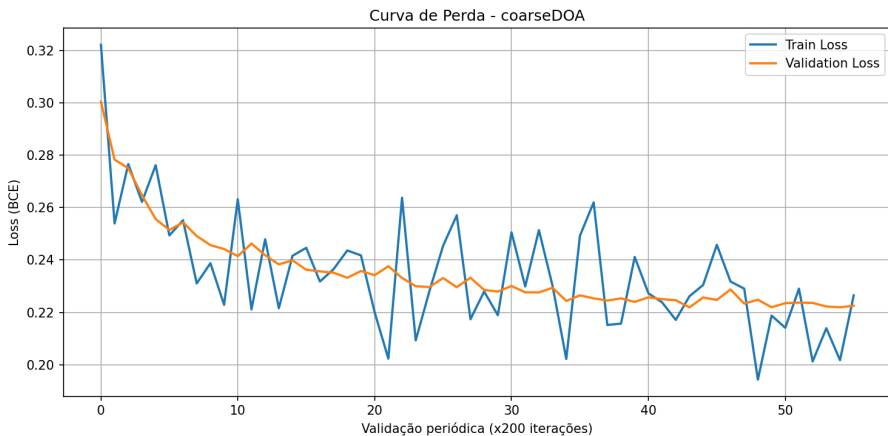


Figura: Avaliação do treinamento.

Curva F1 x Threshold (validação)

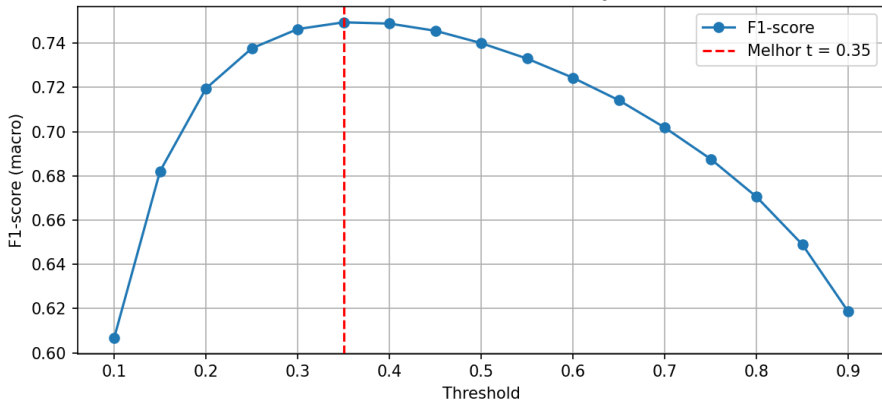


Figura: Escolha do melhor Threshold

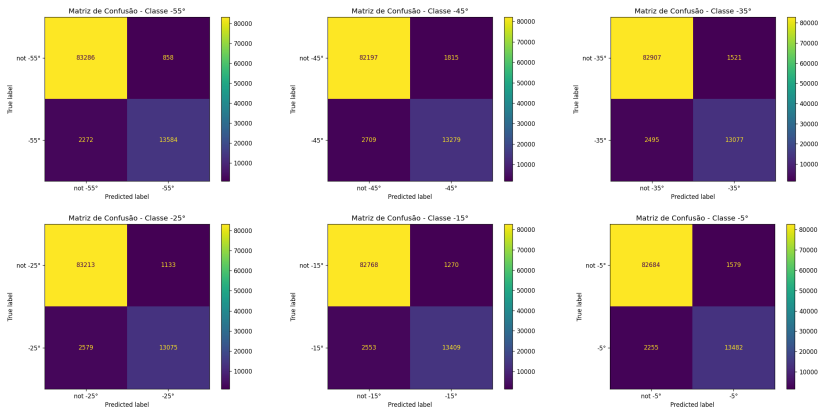


Figura: Matrizes de confusão de cada classe usadas para avaliar a qualidade da predição da rede.

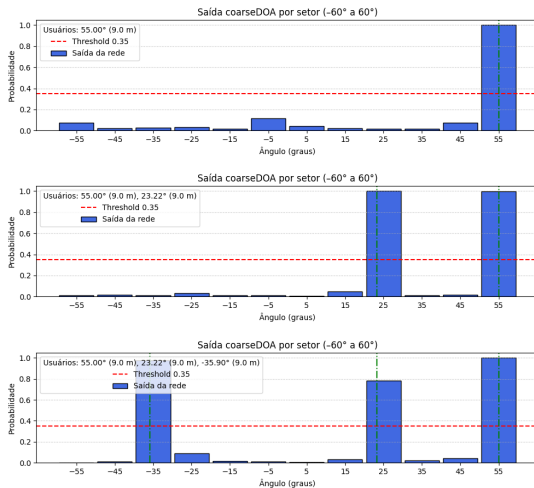


Figura: Vetor de probabilidades obtido na saída da rede do primeiro estágio para uma SNR de 10 dB e 5 amostras

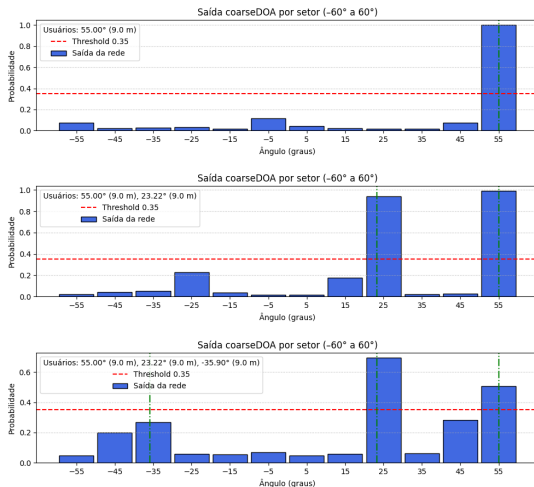


Figura: Vetor de probabilidades obtido na saída da rede do primeiro estágio para uma SNR de 0 dB com 5 amostras.